

Mise en oeuvre d'une API native

Projet Linux Natif - Volet Logiciel - Expression des besoins

Contexte général

Nature du projet

Dans la lignée du volet infrastructure, vous êtes chargé de concevoir et développer une API permettant d'interagir avec le Docker Engine pour exposer des fonctionnalités similaires à l'outil Docker-Compose, en permettant par extension de ces fonctionnalités la modélisation, le déploiement, la supervision et la gestion du cycle de vie d'infrastructures logicielles multi-conteneurs.

Cette API devra pouvoir être utilisée par la suite par toute application front-end, web ou client-lourd, donnant une visualisation graphique et interactive des infrastructures logicielles multi-conteneurs.

Livrables / Attendus

Utilisez une plateforme de versionnage pour votre code source : github, gitlab ou autre et fournissez le lien vers votre dépôt en guise de rendu.

Par ailleurs, votre code source doit fournir les modalités de compilation et de construction de l'image finale de l'API, qui doit être déployée dans un conteneur Docker.

Conditions particulières applicables au travail

Vous pourrez mener le travail en binôme si vous le souhaitez, aux mêmes conditions que pour le volet infrastructure.

Environnement technique

Docker fournit un SDK pour interagir avec le Docker-Engine, celui-ci est disponible en python et en Go. Puisqu'il s'agit de fournir une API native, vous utiliserez le SDK écrit en Go et serez donc contraints à l'utilisation du langage Go.

Bien que Golang fournisse dans son cœur fonctionnel une implémentation TCP/IP convaincante, ainsi qu'un module HTTP, le développement d'une API HTTP REST demande un peu plus qu'un simple stub HTTP. Il est donc conseillé d'utiliser un framework ou une librairie permettant de vous simplifier le processus de développement : [Fiber](#) ou [Gin](#) constituent de bons candidats à ce rôle.

L'API que vous allez développer constitue une alternative à Docker-Compose en l'augmentant. Il est donc important de vous questionner sur le modèle de données. Vous êtes libres de choisir la structure de données adaptée au projet :

- Relationnel
- Non-relationnel

Le mode de stockage des données est aussi à considérer :

- Serveur de bases de données, ce qui implique d'avoir un conteneur additionnel.
- Fichier, ce qui suppose de mettre en place un volume de données persistant pour stocker les configurations et données manipulées par l'API.

Dans tous les cas, à vous d'imaginer la structure générale des données permettant de couvrir le cadre conceptuel et fonctionnel du projet.

Cadre conceptuel et fonctionnel

Cadre conceptuel

Votre API devra manipuler un ensemble d'objets en correspondance avec ce que propose Docker-Compose et le Docker-Engine :

Concepts issus de docker-compose / docker-engine

- Image
- Service
- Conteneur
- Réseau (Network)
- Volume
- Secret
- Label
- Variable d'environnement

Concepts additionnels couverts par l'API

- Serveur
- Projet
- Déploiement

Un **serveur** correspond à une instance du Docker-Engine, qu'elle soit locale ou distante. Elle est caractérisée par une adresse et des modalités d'accès. Pour simplifier, nous considérons un seul "serveur", correspondant à votre environnement docker, auquel l'API aura accès.

Un **projet** est une description d'infrastructure logicielle multi-conteneurs. Un projet couvre donc la description normalement offerte par un fichier docker-compose : description des services, des réseaux privés, des volumes, des secrets, des labels (métadonnées) et des variables d'environnement.

Un **déploiement** est associé à un projet et correspond à sa matérialisation sur un serveur donné. Autrement dit, si le projet décrit une infrastructure logicielle à la manière d'un fichier docker-compose, le déploiement correspond au résultat de l'opération docker-compose up, c'est-à-dire à l'instanciation des divers éléments constitutifs d'un projet, à ceci près qu'elle peut s'effectuer sur un docker-engine distant.

TL;DR : Le projet porte la description de l'infrastructure et le déploiement correspond à son instanciation sur un serveur docker donné.

Cadre fonctionnel

L'API doit fournir les fonctionnalités nécessaires pour créer des projets et en manipuler les éléments constitutifs, en interagissant avec le Docker-Engine. Sur cette base, chaque projet peut être instancié en de multiples déploiements, chacun pouvant porter des configurations particulières.

Par exemple, reprenant le cadre du volet infrastructure du module Linux Avancé, il devrait être possible de créer un projet de déploiement WordPress puis de lui appliquer des configurations spécifiques pour chaque déploiement (nom de domaine différent, paramètres de base de données différents etc.)

Il est donc important de conserver ce qui est spécifique à l'environnement dans les déploiements et de faire en sorte que les projets ne soient que des représentations abstraites d'infrastructures logicielles.

Le **cycle de vie** et la **supervision** des déploiements sont importants à considérer : quel est l'état de santé des conteneurs adossés aux services décrits dans le projet ? C'est un élément que nous devons pouvoir visualiser, sur la base des informations renvoyées par le Docker-Engine.

Docker-Compose permet de gérer le nombre d'instances (conteneurs) correspondant au service. Il est donc aussi important d'imaginer dès le départ une possibilité de scaling (mise à l'échelle) des déploiements.

Il faut donc envisager le cadre fonctionnel sous la forme de plusieurs ensembles fonctionnels :

1. Description d'un projet, par ajout d'éléments descriptifs à la manière de docker-compose
2. Instanciation (déploiement) d'un projet : nécessite de spécifier les éléments variables et spécifiques au déploiement (et marqués comme tels dans la description du projet)
3. Gestion des serveurs, permettant d'interagir avec plusieurs docker-engine et de leur associer les déploiements.

Remarque complémentaire

Certains services instanciés nécessitent l'exposition de ports réseau spécifiques, il faut donc pouvoir, à la manière de docker-compose, spécifier ces mappings au moment du déploiement.

Certains services dépendent d'autres services (une application cliente d'un serveur de bases de données requiert le démarrage préalable dudit serveur de bases de données). Un système de dépendance existe dans docker compose, vous pouvez vous en inspirer.

Roadmap produit

Phase 1 - MVP / POC

Le MVP / POC doit couvrir le périmètre suivant :

- Interaction avec le docker-engine local
- Description d'un projet
- Déploiement d'un projet sur le docker-engine local
- Etat d'un déploiement (running / not-running / partially-running)

Il est requis à ce stade de gérer :

- Services / Conteneurs
- Images (pull, build)
- Volumes
- Variables d'environnement (pour la phase d'instanciation)

Il n'est pas requis à ce stade de gérer les éléments suivants :

- Réseaux
- Secrets
- Labels
- Scaling (multiples instances d'un service)

Phase 2 - POC on steroids : multi-engine

La phase 2 consiste à ajouter la possibilité de déployer les projets sur de multiples serveurs, en ajoutant le scaling à l'ensemble. On reste à ce stade dans la logique : 1 déploiement = 1 serveur cible. Les déploiements composites (1 déploiement sur plusieurs serveurs) sont trop complexes à gérer en l'état et ne sont pas à considérer dans le cadre du projet.

Phase 3 - We're getting there !

Cette phase intègre ce qui manque à la phase 2 :

- Support des réseaux permettant le déploiement de projets multi-réseaux et multi-conteneurs, donc multicouche dans leur architecture
- Support des secrets, permettant la configuration et l'injection de données sensibles au sein des conteneurs pour correspondre aux fonctionnalités de docker-compose
- Support des métadonnées (labels) afin de permettre le fonctionnement de certaines applications en faisant usage (Traefik par exemple).

Autres considérations

Sécurité de l'API

Il peut être intéressant d'implémenter la sécurité de l'API avec la même considération que toute autre API : utilisation de credentials, modèles de sécurité avec module AAA.

Journalisation

Comme toute application qui se respecte, il est important de conserver les traces d'exécution de l'API. Un système de journalisation doit être considéré dès le départ et au moins fournir les traces de debug en cas d'erreur.

Par extension, la traçabilité des appels sur l'API est utile dans un contexte de sécurité plus général.